

PROYECTO SEMESTRAL

Entrega 2

Funcionalidades y priorización

Aquí se listan los requisitos funcionales identificados para Student Organizer y la prioridad 1 corresponde a los requisitos indispensables para el funcionamiento del sistema (se implementan primero) y la prioridad 2 y 3 son funcionalidades complementarias.

#	Requisito funcional	Prioridad
RF01	Iniciar sesión con usuario y contraseña	1
RF02	Ver el perfil del estudiante	2
RF03	Registrar nuevas calificaciones	1
RF04	Consultar todas las calificaciones registradas	1
RF05	Ordenar las calificaciones de mayor a menor	2
RF06	Agregar comentarios o áreas de mejora a cada calificación	3
RF07	Registrar tareas o asignaciones	1
RF08	Asignar una fecha de entrega a cada tarea	1
RF09	Establecer la prioridad de la tarea (Alta, Media o Baja)	2
RF10	Marcar tareas como completadas	1
RF11	Consultar las tareas pendientes	1
RF12	Calcular el promedio general de las calificaciones	1
RF13	Mostrar la mejor y la peor calificación obtenida	2

Requisitos a implementar en esta entrega (prioridad 1): RF01, RF03, RF04, RF07, RF08, RF10, RF11, RF12 son los indispensables para que el sistema funcione y también el (login, gestión de calificaciones y tareas, promedio) y el resto (prioridad 2 y 3) queda para fases posteriores

Consideraciones y requisitos de seguridad

Aquí se listan los requisitos funcionales identificados para Student Organizer y la prioridad 1 corresponde a los requisitos indispensables para el funcionamiento del sistema (se implementan primero) y la prioridad 2 y 3 son funcionalidades complementarias.

- Las contraseñas de los usuarios deben almacenarse cifradas y nunca en texto plano.
- Cada estudiante solo puede consultar y modificar su propia información pues el sistema debe aislar los datos por sesión de usuario.
- El sistema debe validar los datos ingresados como: (campos vacíos, formatos incorrectos en fechas o calificaciones) antes de que se guarden
- El sistema debe cerrar la sesión automáticamente después de un periodo de inactividad
- El acceso a las funciones del sistema debe requerir una sesión autenticada válida (no se debe poder acceder a datos sin haber iniciado sesión)
- La información académica del estudiante debe protegerse contra pérdida mediante respaldos periódicos del archivo/fuente de datos

Nombre: Andreh Callejas Carné: 261828

Fecha	Inicio	Fin	Interrup. (min)	Delta (min)	Tarea	Comentarios
10/08/26	15:00	16:00	0	60	Requisitos funcionales	
11/08/26	14:00	14:40	0	40	Priorización y fase de entrega	
12/08/26	15:40	17:30	30	80	Consideraciones de seguridad	

Prototipos

Login



000

Ingresa a tu perfil

Student Organizer

USUARIO

LUISA234

CONTRASEÑA

▼

Ingresar

The login screen features a dark background with a vertical gradient. On the right, there is a circular illustration of hands organizing a calendar and sticky notes, labeled 'Student Organizer'. The login form is centered, with fields for username and password, and an 'Ingresar' button at the bottom.

Profile



000

Bienvenido Luisa1234

IR a calificaciones

ir a tareas

LUIS ANGEL VALIENTE OSORIO
CARNE: 281288

CIENCIAS DE LA COMPUTACION

The profile screen has a dark background. It displays the user's name 'Luisa1234' and a placeholder for a profile picture. Below the name are two buttons: 'IR a calificaciones' and 'ir a tareas'. To the right, the user's full name and ID number are shown, along with the course name 'CIENCIAS DE LA COMPUTACION'.

Calificaciones



000

Bienvenido a Calificaciones

Calificacion mas baja

Ordenar

Promedio general

Mejor Calificacion

Grades

79.02%

Base on graded assignments

Show What If Score

Assignments

Reading Assignment 23.00 15.00

Plant Genetics Paper 18.00 5

Research Paper 23.00 25.00

Class Journal Week 4 23.00 40.00

Dashboard

The 'Calificaciones' screen shows a welcome message and four buttons for different views: 'Calificacion mas baja', 'Ordenar', 'Promedio general', and 'Mejor Calificacion'. A modal window titled 'Grades' is open, displaying a bar chart with a total score of 79.02%. It lists assignments with their scores and weights.

Registro de Tarea



000

Registrar nueva calificacion?

CURSO

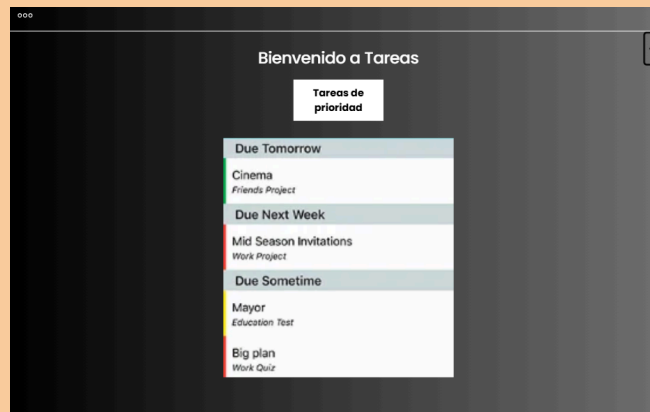
NOTA

COMENTARIO(OPCIONAL)

Guardar nota

The 'Registro de Tarea' screen has a dark background. It prompts the user to 'Registrar nueva calificacion?'. Below this are three input fields for 'CURSO', 'NOTA', and 'COMENTARIO(OPCIONAL)'. A 'Guardar nota' button is positioned to the right of the input fields.

Tareas



Registrar Tarea



Retroalimentación

Se tomó los comentarios de 2 estudiantes de 2 diferentes universidades acerca de los prototipos actuales de Student Organizer:

- Retroalimentación 1: se sugirió un interfaz más atractiva para el usuario para que la navegación de cualquier estudiante dentro del sistema sea mucho más fácil y no surja ningún problema.
- Retroalimentación 2: Está bien implementada la idea, tomando precaución a que no le falte ninguna herramienta de filtrado.

Planificación

- Listar requisitos funcionales. Identificar todas las opciones con las que interactúa el usuario. Horas estimadas: 2. Responsable: Andreh.
- Priorizar requisitos. Asignar prioridad 1, 2 o 3 según qué tan esencial es cada requisito. Horas estimadas: 1. Responsable: Andreh.
- Revisar requisitos de seguridad. Definir consideraciones de seguridad, como validación de login y datos por usuario. Horas estimadas: 1. Responsable: Andreh.
- Identificar las clases necesarias. Detectar las clases del sistema usando el patrón Modelo-Vista-Controlador. Horas estimadas: 3. Responsable: Manuel.
- Organizar según MVC. Confirmar que cada clase respete su rol de Modelo, Vista o Controlador. Horas estimadas: 1. Responsable: Manuel.
- Investigar la tecnología a usar. Comparar herramientas y lenguajes para implementar la solución y seleccionar una. Horas estimadas: 3. Responsable: Fabricio.
- Diseñar wireframes. Pantallas de Login, Perfil, Calificaciones, Tareas y sus formularios. Horas estimadas: 1.5. Responsable: Luis.

Requisito: Autenticar (Login) — Prioridad 1

- Crear archivo .csv de usuarios. Definir la estructura del archivo donde se guardarán los usuarios. Horas estimadas: 1. Responsable: Andreh. Fecha probable: 17 agosto 2026.
- Implementar persistencia de usuarios en PersistenciaDatos. Leer y escribir la información de usuarios almacenada en el archivo .csv. Horas estimadas: 3. Responsable: Andreh. Fecha probable: 21 agosto 2026.
- Implementar clase Usuario. Crear sus atributos y los métodos necesarios para representar y autenticar al estudiante. Horas estimadas: 3. Responsable: Manuel. Fecha probable: 24 agosto 2026.
- Implementar ControladorLogin. Comunicar la Vista con Usuario y controlar el inicio y cierre de sesión. Horas estimadas: 3. Responsable: Fabricio. Fecha probable: 28 agosto 2026.
- Implementar funcionalidad de autenticación en Vista. Permitir ingresar usuario y contraseña y mostrar el resultado del inicio de sesión. Horas estimadas: 3. Responsable: Andreh. Fecha probable: 31 agosto 2026.

Requisito: Ver perfil del estudiante — Prioridad 2

- Implementar funcionalidad de perfil en Vista. Mostrar la información correspondiente al estudiante autenticado y los accesos a las funcionalidades principales. Horas estimadas: 4. Responsable: Manuel. Fecha probable: 11 septiembre 2026.

Requisito: Registrar y consultar calificaciones — Prioridad 1

- Crear archivo .csv de calificaciones. Definir la estructura del archivo donde se guardarán las calificaciones. Horas estimadas: 1. Responsable: Fabricio. Fecha probable: 14 septiembre 2026.
- Implementar persistencia de calificaciones en PersistenciaDatos. Leer y escribir las calificaciones almacenadas en el archivo .csv. Horas estimadas: 3. Responsable: Fabricio. Fecha probable: 18 septiembre 2026.

- Implementar Calificacion y GestorCalificaciones. Representar las calificaciones y permitir agregarlas, ordenarlas, calcular el promedio y obtener la mejor y peor calificación. Horas estimadas: 4. Responsable: Andreh. Fecha probable: 21 septiembre 2026.
- Implementar ControladorCalificaciones. Coordinar el registro, consulta, ordenamiento y operaciones relacionadas con las calificaciones. Horas estimadas: 4. Responsable: Manuel. Fecha probable: 25 septiembre 2026.
- Implementar registro de calificaciones en Vista. Permitir ingresar curso, nota y comentario o área de mejora. Horas estimadas: 3. Responsable: Fabricio. Fecha probable: 28 septiembre 2026.
- Implementar consulta de calificaciones en Vista. Mostrar las calificaciones, promedio general, mejor nota y peor nota. Horas estimadas: 4. Responsable: Andreh. Fecha probable: 2 octubre 2026.

Requisito: Registrar y consultar tareas — Prioridad 1

- Crear archivo .csv de tareas. Definir la estructura del archivo donde se guardarán las tareas. Horas estimadas: 1. Responsable: Manuel. Fecha probable: 9 octubre 2026.
- Implementar persistencia de tareas en PersistenciaDatos. Leer, escribir y actualizar las tareas almacenadas en el archivo .csv. Horas estimadas: 3. Responsable: Manuel. Fecha probable: 12 octubre 2026.
- Implementar clase Tarea. Crear sus atributos y los métodos necesarios para manejar fecha de entrega, prioridad y estado de completada. Horas estimadas: 4. Responsable: Fabricio. Fecha probable: 16 octubre 2026.
- Implementar ControladorTareas. Coordinar el registro de tareas, consulta de pendientes y cambio de estado de las tareas completadas. Horas estimadas: 4. Responsable: Andreh. Fecha probable: 19 octubre 2026.
- Implementar registro de tareas en Vista. Permitir ingresar nombre, fecha de entrega y prioridad. Horas estimadas: 3. Responsable: Manuel. Fecha probable: 23 octubre 2026.
- Implementar consulta de tareas pendientes en Vista. Mostrar las tareas pendientes, su prioridad y permitir marcarlas como completadas. Horas estimadas: 4. Responsable: Fabricio. Fecha probable: 26 octubre 2026.

Nombre: Luis Angel valiente Osorio / Carne 261288

Fecha	Inicio	Fin	Tiempo interrupción	Delta tiempo	Tarea	Comentarios
10/8/2026	6:00	8:00	10	110	Prototipado	
11/8/2026	7:10	8:00	10	40	Retroalimentación de prototipado	
12/8/2026	6:25	8:30	5	120	Planificación	

Identificación y descripción de clases

Student Organizer utiliza el patrón Modelo–Vista–Controlador (MVC). Las clases Modelo representan los datos y realizan las operaciones del sistema, los Controladores coordinan las acciones entre el modelo y la interfaz, y la Vista se encarga de recibir información del usuario y mostrar los resultados.

Modelo

Usuario

Propósito: representar al estudiante registrado en el sistema y almacenar la información necesaria para iniciar sesión.

Atributos:

- nombreUsuario: nombre utilizado para identificar al usuario.
- contraseña: contraseña asociada al usuario.
- carne: identificación académica del estudiante.

Métodos:

- Usuario(nombreUsuario, contraseña, carne): constructor.
- autenticarse(usuario, contraseña): comprueba las credenciales ingresadas.
- getters y setters: permiten acceder y modificar los atributos.
- toString(): devuelve una representación textual del usuario.

Calificacion

Propósito: representar una calificación obtenida por el estudiante.

Atributos:

- curso: nombre del curso.
- nota: calificación obtenida.
- comentario: comentario o área de mejora asociada.

Métodos:

- Calificacion(curso, nota, comentario): constructor.
- getters y setters: permiten acceder y modificar los atributos.
- toString(): devuelve una representación textual de la calificación.

GestorCalificaciones

Propósito: administrar las calificaciones registradas y realizar operaciones sobre ellas.

Atributos:

- listaCalificaciones: colección de calificaciones registradas.

Métodos:

- GestorCalificaciones(): constructor.
- agregar(calificacion): agrega una nueva calificación.
- ordenarDescendente(): ordena las calificaciones de mayor a menor.
- calcularPromedio(): calcula el promedio general.
- obtenerMejor(): devuelve la mejor calificación.
- obtenerPeor(): devuelve la peor calificación.
- getters y setters.
- toString(): devuelve una representación textual del gestor.

Tarea

Propósito: representar una tarea o asignación académica.

Atributos:

- nombre: nombre de la tarea.
- fechaEntrega: fecha en la que debe entregarse.
- prioridad: prioridad Alta, Media o Baja.
- completada: indica si la tarea fue completada.

Métodos:

- Tarea(nombre, fechaEntrega, prioridad): constructor.
- marcarCompletada(): cambia el estado de la tarea a completada.
- getters y setters.
- toString(): devuelve una representación textual de la tarea.

PersistenciaDatos

Propósito: almacenar y recuperar localmente la información de usuarios, calificaciones y tareas mediante archivos CSV.

Atributos:

- rutaUsuarios: ubicación del archivo de usuarios.
- rutaCalificaciones: ubicación del archivo de calificaciones.
- rutaTareas: ubicación del archivo de tareas.

Métodos:

- PersistenciaDatos(): constructor.
- leerUsuarios(): recupera los usuarios almacenados.
- guardarUsuario(usuario): almacena un usuario.
- leerCalificaciones(): recupera las calificaciones.
- guardarCalificacion(calificacion): almacena una calificación.
- leerTareas(): recupera las tareas.
- guardarTarea(tarea): almacena una tarea.
- actualizarTarea(tarea): actualiza la información de una tarea.
- getters y setters.
- toString(): devuelve una representación textual de la configuración de almacenamiento

Controlador

ControladorLogin

Propósito: controlar el inicio y cierre de sesión y comunicar la Vista con Usuario.

Atributos:

- usuarioActual: usuario que actualmente tiene una sesión iniciada.

Métodos:

- ControladorLogin(): constructor.
- iniciarSesion(usuario, contraseña): comprueba las credenciales del usuario.
- cerrarSesion(): finaliza la sesión actual.
- getUsuarioActual(): devuelve el usuario autenticado.
- toString(): devuelve una representación textual del controlador.

ControladorCalificaciones

Propósito: coordinar las operaciones relacionadas con las calificaciones.

Atributos:

- gestorCalificaciones: referencia al gestor de calificaciones.

Métodos:

- ControladorCalificaciones(gestorCalificaciones): constructor.
- registrarCalificacion(curso, nota, comentario): registra una calificación.
- consultarCalificaciones(): obtiene las calificaciones registradas.
- ordenarCalificaciones(): solicita ordenar las calificaciones.
- calcularPromedio(): obtiene el promedio general.
- obtenerMejor(): obtiene la mejor calificación.
- obtenerPeor(): obtiene la peor calificación.
- getters y setters.
- toString(): devuelve una representación textual del controlador.

ControladorTareas

Propósito: coordinar el registro, consulta y actualización de las tareas.

Atributos:

- listaTareas: colección de tareas registradas.

Métodos:

- ControladorTareas(): constructor.
- registrarTarea(nombre, fechaEntrega, prioridad): registra una nueva tarea.
- consultarPendientes(): obtiene las tareas que no han sido completadas.
- marcarCompletada(tarea): marca una tarea como completada.
- getters y setters.
- toString(): devuelve una representación textual del controlador.

Vista

Vista

Propósito: manejar la interacción con el estudiante. Se encarga de recibir los datos ingresados por el usuario y mostrar los resultados proporcionados por los controladores. No contiene lógica de negocio.

Atributos:

- controladorLogin: referencia al controlador de inicio de sesión.
- controladorCalificaciones: referencia al controlador de calificaciones.
- controladorTareas: referencia al controlador de tareas.

Métodos:

- Vista(controladorLogin, controladorCalificaciones, controladorTareas): constructor.
- mostrarLogin(): muestra las opciones para iniciar sesión.
- mostrarPerfil(): muestra la información del estudiante.
- mostrarCalificaciones(): muestra las opciones y resultados relacionados con calificaciones.
- mostrarTareas(): muestra las opciones y resultados relacionados con tareas.
- mostrarMensaje(mensaje): muestra información o resultados al usuario.
- toString(): devuelve una representación textual de la vista.

Organización según Modelo–Vista–Controlador

Modelo:

Usuario, Calificacion, GestorCalificaciones, Tarea y PersistenciaDatos.

Controlador:

ControladorLogin, ControladorCalificaciones y ControladorTareas.

Vista:

Vista.

La Vista se utiliza únicamente para obtener información del usuario y mostrar resultados. Los controladores reciben las solicitudes de la Vista y coordinan las operaciones con las clases Modelo. Las clases Modelo no muestran resultados directamente al usuario.

Nombre: [MANUEL RIO]

Carné: [16639]

Fecha	Inicio	Fin	Interrup. (min)	Delta (min)	Tarea	Comentarios
[12/08/2026]	[7:00]	[9:00]	[0]	[120]	Identificar las clases necesarias	Con ayuda de compañeros
12/08/2026]	[9:00]	[10:30]	[10]	[80]	Organizar las clases según MVC	Con ayuda de compañeros

Investigación de tecnología

Para el desarrollo de Student Organizer se investigaron las herramientas y tecnologías necesarias para implementar una aplicación de escritorio que permita gestionar información académica del estudiante.

Java

Se seleccionó Java como lenguaje de programación principal. Java permite desarrollar el sistema utilizando el paradigma de programación orientada a objetos, mediante la creación de clases, objetos, atributos y métodos. Esto permite implementar la estructura Modelo–Vista–Controlador planteada para Student Organizer.

El proyecto será desarrollado mediante archivos con extensión .java, donde se implementarán las diferentes clases identificadas durante el diseño del sistema.

Visual Studio Code

Se utilizará Visual Studio Code como entorno de desarrollo para escribir, organizar y modificar los archivos .java que conforman el proyecto. Esta herramienta permite trabajar con los diferentes archivos del programa desde un mismo espacio de trabajo.

Almacenamiento local

La información utilizada por Student Organizer será almacenada localmente en la computadora. De acuerdo con la planificación actual del proyecto, se utilizarán archivos .csv para almacenar información como usuarios, calificaciones y tareas.

Los archivos .java serán responsables de leer, procesar y modificar la información almacenada según las funcionalidades del sistema.

Tecnología seleccionada

Para la implementación de Student Organizer se seleccionó Java como lenguaje de programación, Visual Studio Code como herramienta de desarrollo y archivos locales como mecanismo de almacenamiento de datos.

Esta combinación permite desarrollar las funcionalidades planteadas sin requerir una base de datos o infraestructura externa y es compatible con la estructura orientada a objetos y el patrón Modelo–Vista–Controlador definidos para el proyecto.

Calendario de planificación

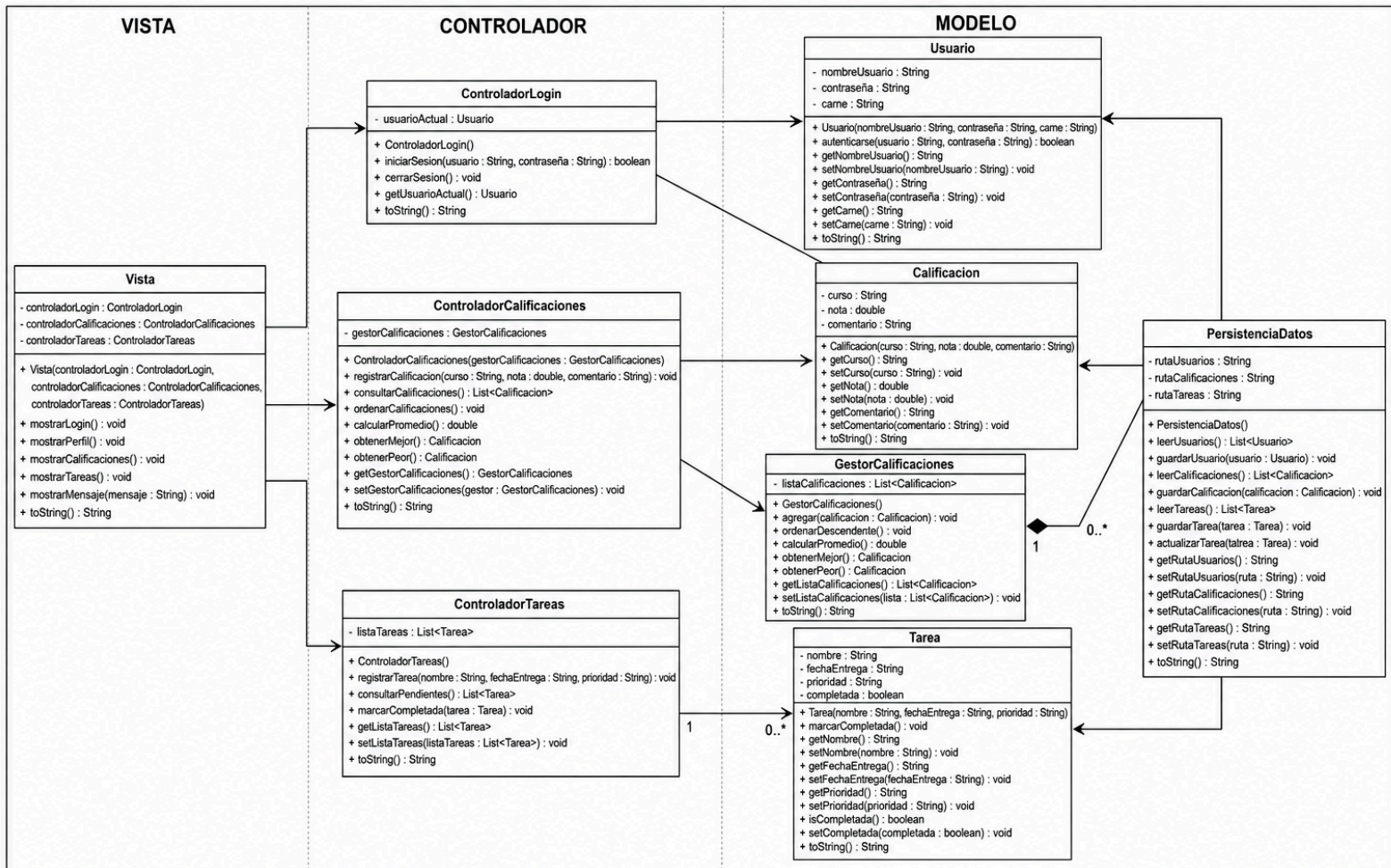
Calendario de planificación — Student Organizer

Planificación hasta final del semestre

Fecha probable	Tarea	Responsable
17 ago 2026	Crear archivo CSV de usuarios	Callejas
21 ago 2026	Implementar persistencia de usuarios en PersistenciaDatos	Callejas
24 ago 2026	Implementar clase Usuario	Manuel
28 ago 2026	Implementar ControladorLogin	Fabrizio
31 ago 2026	Implementar autenticación en Vista	Callejas
4 sep 2026	Probar e integrar sistema de login	Todos
11 sep 2026	Implementar funcionalidad de perfil en Vista	Manuel
14 sep 2026	Crear archivo CSV de calificaciones	Fabrizio
18 sep 2026	Implementar persistencia de calificaciones en PersistenciaDatos	Fabrizio
21 sep 2026	Implementar Calificacion y GestorCalificaciones	Callejas
25 sep 2026	Implementar ControladorCalificaciones	Manuel
28 sep 2026	Implementar registro de calificaciones en Vista	Fabrizio
2 oct 2026	Implementar consulta de calificaciones en Vista	Callejas
5 oct 2026	Integrar y probar módulo de calificaciones	Todos
9 oct 2026	Crear archivo CSV de tareas	Manuel
12 oct 2026	Implementar persistencia de tareas en PersistenciaDatos	Manuel
16 oct 2026	Implementar clase Tarea	Fabrizio
19 oct 2026	Implementar ControladorTareas	Callejas
23 oct 2026	Implementar registro de tareas en Vista	Manuel
26 oct 2026	Implementar consulta de tareas pendientes en Vista	Fabrizio
30 oct 2026	Integrar y probar módulo de tareas	Todos
6 nov 2026	Implementar funcionalidades secundarias pendientes	Todos
13 nov 2026	Integración completa de Student Organizer	Todos
20 nov 2026	Mostrar sistema a usuarios finales y recopilar retroalimentación	Todos
27 nov 2026	Corregir errores y aplicar cambios según retroalimentación	Todos
Fin del semestre	Pruebas finales y preparación de la entrega	Todos

UML

Diagrama de clases UML — Student Organizer



Nombre: Fabricio Xiloj

Carné: 262140

Fecha	Inicio	Fin	Interrup. (min)	Delta (min)	Tarea	Comentarios
09/08/26	17:00	19:00	40	80	Investigacion de tecnologia	se decidió usar lo más básico para este proyecto
11/08/26	18:00	18:40	0	40	Elaboración del diagrama UML	
12/08/26	16:00	17:30	0	90	Elaboración del calendario de planificación	Son fechas estimadas, más no exactas.